

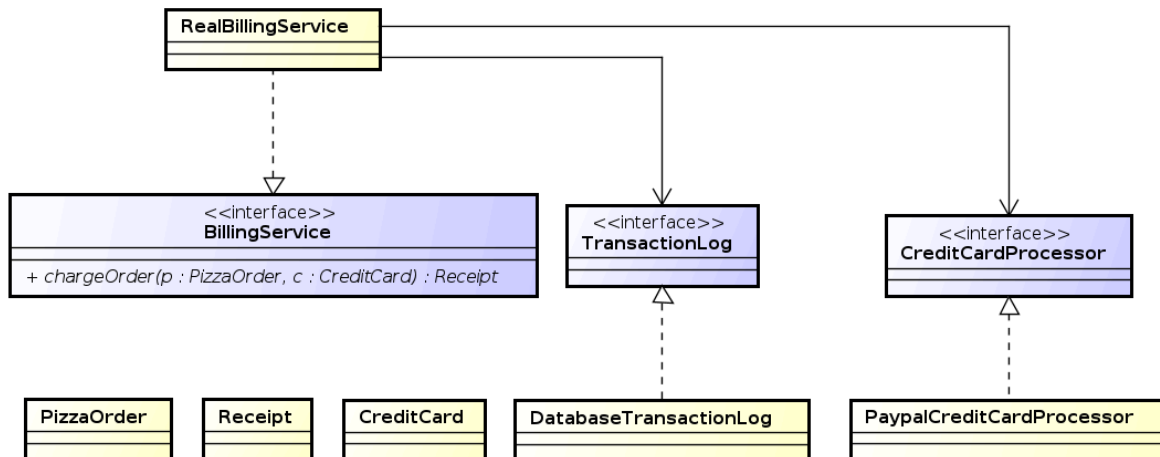
Садржај

1. Мотивација

1.1. Мотивација

- Објекти иоле сложенијих апликација формирају сложене графове зависности.
- Како објекат “добија” референце на зависне објекте?

1.2. Мотивација



1.3. Класичан приступ добављања референци

```
public class RealBillingService implements BillingService {

    @Override
    public Receipt chargeOrder(PizzaOrder order, CreditCard creditCard) {
        CreditCardProcessor processor = new PaypalCreditCardProcessor();
        TransactionLog transactionLog = new DatabaseTransactionLog();

        try {
            ChargeResult result = processor.charge(creditCard, order.getAmount());
            transactionLog.logChargeResult(result);

            return result.isSuccessful()
                ? Receipt.forSuccessfulCharge(order.getAmount())
                : Receipt.forDeclinedCharge(result.getDeclineMessage());
        } catch (UnreachableException e) {
            transactionLog.logConnectException(e);
            return Receipt.forSystemFailure(e.getMessage());
        }
    }
}
```

1.4. Употреба Singleton/Factory обрасца

Објекат се сам брине о добављању референци али то чини посредством глобалне дељене референце.

```

public class RealBillingService implements BillingService {

    public Receipt chargeOrder(PizzaOrder order, CreditCard creditCard) {
        CreditCardProcessor processor = CreditCardProcessorFactory.getInstance();
        TransactionLog transactionLog = TransactionLogFactory.getInstance();

        try {
            ChargeResult result = processor.charge(creditCard, order.getAmount());
            transactionLog.logChargeResult(result);

            return result.isSuccessful() ?
                Receipt.forSuccessfulCharge(order.getAmount()) :
                Receipt.forDeclinedCharge(result.getDeclineMessage());
        } catch (UnreachableException e) {
            transactionLog.logConnectException(e);
            return Receipt.forSystemFailure(e.getMessage());
        }
    }
}

```

1.5. Singleton/Factory - тестирање

```

public class RealBillingServiceTest extends TestCase {

    private final PizzaOrder order = new PizzaOrder(100);
    private final CreditCard creditCard = new CreditCard(5000);
    private final InMemoryTransactionLog transactionLog = new
InMemoryTransactionLog();
    private final FakeCreditCardProcessor creditCardProcessor = new
FakeCreditCardProcessor();

    @Override
    public void setUp() {
        TransactionLogFactory.setInstance(transactionLog);
        CreditCardProcessorFactory.setInstance(creditCardProcessor);
    }

    @Override
    public void tearDown() {
        TransactionLogFactory.setInstance(null);
        CreditCardProcessorFactory.setInstance(null);
    }

    public void testSuccessfulCharge() {
        RealBillingService billingService = new RealBillingService();
        Receipt receipt = billingService.chargeOrder(order, creditCard);

        assertTrue(receipt.hasSuccessfulCharge());
        assertEquals(100.0, receipt.getAmount(), 0.001);
        assertEquals(creditCard, creditCardProcessor.getCardOfOnlyCharge());
        assertEquals(100.0, creditCardProcessor.getAmountOfOnlyCharge(), 0.001);
        assertTrue(transactionLog.wasSuccessLogged());
    }
}

```

1.6. Употреба Singleton/Factory обрасца - проблеми

- Дељена референца - морамо посебно да пазимо да је постављамо на праве вредности.

- Немогуће паралелизовати тестове.

2. *Dependency Injection*

2.1. Уметање зависности - *Dependency Injection*

- Измештање надлежности за добављање референци ван објекта - неко други ће се бринути да “уметне” референце пре њихове употребе.
- Предности:
 - Код се поједностављује. Зависност између класа је базирана на апстрактним интерфејсима што позитивно утиче на одржавање (*maintability*), поновну искористљивост (*reusability*) и поделу посла и надлежности.
 - Објекат ће до тренутка позива његових сервисних метода већ бити на одговарајући начин иницијализован. Смањује се тзв. *boilerplate* код.
 - Тестирање је далеко једноставније. Креирање “лажних” објеката (*mockup*) је могуће и једноставно се изводи. Могућа паралелизација тестова.

2.2. Механизми уметања зависности

- Путем параметара конструктора.
- Путем мутатор метода (*setters*).
- Путем имплементираног интерфејса.

2.3. Инјекција путем параметара конструктора¹

```
Client(Service service) {  
    this.service = service;  
}
```

2.4. Инјекција путем *setter* метода¹

```
public void setService(Service service) {  
    this.service = service;  
}
```

2.5. Инјекција путем интерфејса¹

```
public interface ServiceSetter {  
    public void setService(Service service);  
}  
  
public class client implements ServiceSetter {  
  
    private Service service;  
  
    @Override  
    public void setService(Service service) {  
        this.service = service;  
    }  
}
```

¹Wikipedia - Dependency Injection: https://en.wikipedia.org/wiki/Dependency_injection

```

public class RealBillingService implements BillingService {
    private final CreditCardProcessor processor;
    private final TransactionLog transactionLog;

    public RealBillingService(CreditCardProcessor processor,
        TransactionLog transactionLog) {
        this.processor = processor;
        this.transactionLog = transactionLog;
    }

    public Receipt chargeOrder(PizzaOrder order, CreditCard creditCard) {
        try {
            ChargeResult result = processor.charge(creditCard, order.getAmount());
            transactionLog.logChargeResult(result);

            return result.isSuccessful() ?
                Receipt.forSuccessfulCharge(order.getAmount()) :
                Receipt.forDeclinedCharge(result.getDeclineMessage());
        } catch (UnreachableException e) {
            transactionLog.logConnectException(e);
            return Receipt.forSystemFailure(e.getMessage());
        }
    }
}

```

2.6. Употреба *DI* - тестирање

```

public class RealBillingServiceTest extends TestCase {

    private final PizzaOrder order = new PizzaOrder(100);
    private final CreditCard creditCard = new CreditCard(5000);

    private final InMemoryTransactionLog transactionLog =
        new InMemoryTransactionLog();
    private final FakeCreditCardProcessor creditCardProcessor =
        new FakeCreditCardProcessor();

    public void testSuccessfulCharge() {
        RealBillingService billingService = new RealBillingService(
            creditCardProcessor, transactionLog);
        Receipt receipt = billingService.chargeOrder(order, creditCard);

        assertTrue(receipt.hasSuccessfulCharge());
        assertEquals(100.0, receipt.getAmount(), 0.001);
        assertEquals(creditCard, creditCardProcessor.getCardOfOnlyCharge());
        assertEquals(100.0, creditCardProcessor.getAmountOfOnlyCharge(), 0.001);
        assertTrue(transactionLog.wasSuccessLogged());
    }
}

```

2.7. *DI* контејнери

- *DI* се може имплементирати и без посебног алата/оквира.
- *DI* контејнери омогућавају наметање одређених конвенција за примену овог обрасца.
- Коришћење *DI* контејера доноси одређене предности:
 - Употреба најбоље праксе

- Стандардизација

2.8. *DI* контејнери за Јаву

- Google Guice
- PicoContainer
- Spring
- ...

2.9. Стандардизација за програмски језик Јава

- JSR-330²
- Дефинише скуп стандардних Јава анотација за *DI*:
 - `Provider<T>` - Provides instances of T
 - `Inject` - Identifies injectable constructors, methods, and fields.
 - `Named` - String-based qualifier.
 - `Qualifier` - Identifies qualifier annotations.
 - `Scope` - Identifies scope annotations.
 - `Singleton` - Identifies a type that the injector only instantiates once.

3. Google Guice

3.1. Google Guice

- *Lightweight* оквир за *DI* у Јави.
- Развијен од стране Google-а.
- Конфигурација базирана на Јава анотацијама.

²<https://code.google.com/p/atinject/>

3.2. Инјекција путем конструктора

```
public class RealBillingService implements BillingService {
    private final CreditCardProcessor processor;
    private final TransactionLog transactionLog;

    @Inject
    public RealBillingService(CreditCardProcessor processor,
        TransactionLog transactionLog) {
        this.processor = processor;
        this.transactionLog = transactionLog;
    }

    public Receipt chargeOrder(PizzaOrder order, CreditCard creditCard) {
        try {
            ChargeResult result = processor.charge(creditCard, order.getAmount());
            transactionLog.logChargeResult(result);

            return result.wasSuccessful()
                ? Receipt.forSuccessfulCharge(order.getAmount())
                : Receipt.forDeclinedCharge(result.getDeclineMessage());
        } catch (UnreachableException e) {
            transactionLog.logConnectException(e);
            return Receipt.forSystemFailure(e.getMessage());
        }
    }
}
```

3.3. Конфигурација за повезивање - *binding/wiring*

```
public class BillingModule extends AbstractModule {
    @Override
    protected void configure() {
        bind(TransactionLog.class).to(DatabaseTransactionLog.class);
        bind(CreditCardProcessor.class).to(PaypalCreditCardProcessor.class);
        bind(BillingService.class).to(RealBillingService.class);
    }
}
```

3.4. Употреба контејнера

```
public static void main(String[] args) {
    Injector injector = Guice.createInjector(new BillingModule());
    BillingService billingService = injector.getInstance(BillingService.class);
    Receipt result = billingService.chargeOrder(new PizzaOrder(100),
                                                new CreditCard(500));
    System.out.println(result.hasSuccessfulCharge());
}
```

3.5. *Linked Bindings*

```
public class BillingModule extends AbstractModule {
    @Override
    protected void configure() {
        bind(TransactionLog.class).to(DatabaseTransactionLog.class);
        bind(DatabaseTransactionLog.class).to(MySqlDatabaseTransactionLog.class);
    }
}
```

3.6. Custom Bindings Annotations

```
package example.pizza;
import com.google.inject.BindingAnnotation;
import java.lang.annotation.Target;
import java.lang.annotation.Retention;
import static java.lang.annotation.RetentionPolicy.RUNTIME;
import static java.lang.annotation.ElementType.PARAMETER;
import static java.lang.annotation.ElementType.FIELD;
import static java.lang.annotation.ElementType.METHOD;

@BindingAnnotation @Target({ FIELD, PARAMETER, METHOD }) @Retention(RUNTIME)
public @interface PayPal {}

...

public class RealBillingService implements BillingService {

    @Inject
    public RealBillingService(@PayPal CreditCardProcessor processor,
        TransactionLog transactionLog) {
        ...
    }
    ...
    bind(CreditCardProcessor.class)
        .annotatedWith(PayPal.class)
        .to(PayPalCreditCardProcessor.class);
}
```

3.7. @Named Binding Annotation

```
public class RealBillingService implements BillingService {

    @Inject
    public RealBillingService(@Named("Checkout") CreditCardProcessor processor,
        TransactionLog transactionLog) {
        ...
    }
    ...
    ...
    bind(CreditCardProcessor.class)
        .annotatedWith(Names.named("Checkout"))
        .to(CheckoutCreditCardProcessor.class);
}
```

3.8. Instance Bindings

```
bind(String.class)
    .annotatedWith(Names.named("JDBC URL"))
    .toInstance("jdbc:mysql://localhost/pizza");
bind(Integer.class)
    .annotatedWith(Names.named("login timeout seconds"))
    .toInstance(10);
```

3.9. @Provides Methods

```
public class BillingModule extends AbstractModule {
    @Override
    protected void configure() {
        ...
    }

    @Provides
    TransactionLog provideTransactionLog() {
        DatabaseTransactionLog transactionLog = new DatabaseTransactionLog();
        transactionLog.setJdbcUrl("jdbc:mysql://localhost/pizza");
        transactionLog.setThreadPoolSize(30);
        return transactionLog;
    }
}
...
@Provides @PayPal
CreditCardProcessor providePayPalCreditCardProcessor(
    @Named("PayPal API key") String apiKey) {
    PayPalCreditCardProcessor processor = new PayPalCreditCardProcessor();
    processor.setApiKey(apiKey);
    return processor;
}
```

3.10. Provider Bindings

```
public class DatabaseTransactionLogProvider
    implements Provider<TransactionLog> {
    private final Connection connection;

    @Inject
    public DatabaseTransactionLogProvider(Connection connection) {
        this.connection = connection;
    }

    public TransactionLog get() {
        DatabaseTransactionLog transactionLog = new DatabaseTransactionLog();
        transactionLog.setConnection(connection);
        return transactionLog;
    }
}
...
public class BillingModule extends AbstractModule {
    @Override
    protected void configure() {
        bind(TransactionLog.class)
            .toProvider(DatabaseTransactionLogProvider.class);
    }
}
```


3.11. *Scopes*

```
@Singleton
public class InMemoryTransactionLog implements TransactionLog {
    /* everything here should be threadsafe! */
}
...
bind(TransactionLog.class)
    .to(InMemoryTransactionLog.class).in(Singleton.class);
...
@Provides @Singleton
TransactionLog provideTransactionLog() {
    ...
}
...
bind(Bar.class).to(Applebees.class).in(Singleton.class);
bind(Grill.class).to(Applebees.class).in(Singleton.class);
```

4. Injector

4.1. Injector

- <https://github.com/alecthomas/injector>
- Python библиотека за *DI* моделирана према *Google Guice* али са *Pythonic API*.

4.2. Једноставан пример

```
>>> from injector import Injector, inject
>>> class Inner(object):
...     def __init__(self):
...         self.forty_two = 42
...
>>> class Outer(object):
...     @inject
...     def __init__(self, inner: Inner):
...         self.inner = inner
...
>>> injector = Injector()
>>> outer = injector.get(Outer)
>>> outer.inner.forty_two
42
```

- Python type hints - <https://docs.python.org/3/library/typing.html>

4.3. Сложенији пример

```
from injector import Key
Name = Key('name')
Description = Key('description')
```

```

from injector import inject, provider, Module

class User(object):
    @inject
    def __init__(self, name: Name, description: Description):
        self.name = name
        self.description = description

class UserModule(Module):
    def configure(self, binder):
        binder.bind(User)

class UserAttributeModule(Module):
    def configure(self, binder):
        binder.bind(Name, to='Sherlock')

    @provider
    def describe(self, name: Name) -> Description:
        return '%s is a man of astounding insight' % name

```

<http://injector.readthedocs.io/en/latest/terminology.html#injection>

4.4. Сложенији пример

```

from injector import Injector
injector = Injector([UserModule(), UserAttributeModule()])

```

или

```

injector = Injector([UserModule, UserAttributeModule])

```

Употреба:

```

>>> injector.get(Name)
'Sherlock'
>>> injector.get(Description)
'Sherlock is a man of astounding insight'

>>> user = injector.get(User)
>>> isinstance(user, User)
True
>>> user.name
'Sherlock'
>>> user.description
'Sherlock is a man of astounding insight'

```

5. Flask injector

5.1. Flask injector

- Веза између injector библиотеке и Flask оквира за развој.

5.2. Пример употребе

```
import sqlite3
from flask import Flask, Config
from flask.views import View
from flask_injector import FlaskInjector
from injector import inject

app = Flask(__name__)

@app.route("/bar")
def bar():
    return render("bar.html")

@app.route("/foo")
@inject(db=sqlite3.Connection)
def foo(db):
    users = db.execute('SELECT * FROM users').all()
    return render("foo.html")

def configure(binder):
    binder.bind(
        sqlite3.Connection,
        to=sqlite3.Connection(':memory:'),
        scope=request,
    )

FlaskInjector(app=app, modules=[configure])

app.run()
```

6. Референце

- [Dependency Injection on Wikipedia](#)³
- Martin Fowler, [Inversion of Control Containers and the Dependency Injection pattern](#)⁴, January 2004.
- [Google Guice Wiki](#)⁵
- [Injector dokumentacija](#)⁶
- [Flask Injector projekat](#)⁷

³https://en.wikipedia.org/wiki/Dependency_injection

⁴<http://www.martinfowler.com/articles/injection.html>

⁵<https://github.com/google/guice/wiki>

⁶<http://injector.readthedocs.io/en/latest/>

⁷https://github.com/alecthoas/flask_injector